

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Pujitha

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	
---------------	-----	--	--	-------------------------------	---------------------------	--

Annexure A

Constraint-Based Problem Solving

1. Scenario: A company wants to deploy a secure access system using face recognition in a low-light environment.

Constraints:

- A. Limited computational resources (embedded device)
- B. Faces may appear at different orientations ($\pm 30^\circ$)
- C. Real-time recognition required (<200 ms per face)

Questions:

1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?
2. Suggest preprocessing steps to improve recognition under low-light conditions.
3. Discuss strategies to optimize computation for real-time performance on constrained hardware.

Question 2: Photo Album Organization

2. Scenario: You are designing an AI system to automatically organize a large personal photo collection by people and events.

Constraints:

- A. Millions of images with inconsistent resolution and quality
- B. Multiple photos per person in different poses
- C. Privacy considerations: all processing must occur locally

Questions:

1. How would you use face recognition and clustering to group photos of the same individual?
2. Propose a method to detect and separate images with occlusions or partial faces.
3. Discuss trade-offs between accuracy and computational efficiency for local processing.

3. Scenario: A smart city surveillance system must detect moving objects and track pedestrians across multiple cameras.

Constraints:

- A. Crowded scenes with frequent occlusion
- B. Limited network bandwidth between cameras
- C. Real-time processing requirement

Questions:

1. How would you design a foreground–background separation algorithm to minimize false positives in crowded scenes?
2. Explain how particle filters can be used to handle occlusion during tracking.
3. Propose an efficient multi-camera strategy for consistent pedestrian identification across viewpoints.

4. Scenario: A self-driving car must navigate urban streets while detecting lanes, road signs, and pedestrians.

Constraints:

- A. Variable weather and lighting conditions (rain, night)
- B. Real-time decision-making (≤ 50 ms per frame)
- C. High-density traffic with moving pedestrians

Questions:

1. How would you implement road detection robust to shadows, rain, and nighttime illumination?
2. Propose a pipeline to simultaneously detect road signs and pedestrians efficiently.
3. Discuss the trade-offs between deep learning-based detectors and classical computer vision methods under real-time constraints.

5. Scenario: An AR system for surgical assistance must overlay segmented organs in real time.

Constraints:

- A. High-resolution 3D medical images (CT/MRI)
- B. Latency must be < 100 ms for surgeon interaction
- C. Limited GPU memory on wearable AR headset

Questions:

1. How would you implement real-time organ segmentation under these memory and latency constraints?
2. Discuss methods to integrate segmentation results accurately in AR/MR overlays.
3. Suggest strategies to reduce computational load while maintaining segmentation accuracy.

Scenario 1: Face Recognition Using Eigenfaces in Low-Light Environment

1. Adapting Eigenface recognition for illumination and orientation

Eigenfaces are sensitive to changes in lighting and face pose. To improve performance:

- **Illumination normalization:** Apply histogram equalization or CLAHE before extracting Eigenfaces.
- **Pose normalization:** Detect facial landmarks such as the eyes and nose, then **align and rotate the face** to a standard position.
- **Handle $\pm 30^\circ$ orientation:** Use several training images of each person at different angles, or generate rotated/pose-normalized face images.
- **Use PCA carefully:** Apply PCA after preprocessing to obtain the most important facial features while reducing the feature dimension.
- **Multiple templates:** Store a few Eigenface representations for each person under different orientations and compare the input with the closest template.

2. Preprocessing for low-light conditions

The following preprocessing pipeline can improve recognition:

Input Image → Face Detection → Illumination Enhancement → Noise Removal → Face Alignment → Resize → Normalization → Eigenface Extraction

Recommended steps:

1. **Face detection** – Detect the face using a lightweight detector.
2. **CLAHE** – Enhances local contrast without excessively increasing noise.
3. **Gamma correction** – Brightens dark facial regions.
4. **Denoising** – Use a small Gaussian or median filter to reduce low-light noise.
5. **Face alignment** – Align the eyes and other landmarks.
6. **Grayscale conversion** – Eigenfaces work with grayscale images.
7. **Resize** – Convert all faces to a small fixed size such as 64×64 pixels.
8. **Intensity normalization** – Normalize pixel values before PCA.

3. Optimizing computation for real-time performance

Since the embedded device has limited resources and recognition must be less than 200 ms, the following strategies can be used:

- **Reduce image resolution:** Use small images such as 64×64 instead of high-resolution images.
- **Reduce PCA dimensions:** Keep only the most important Eigenfaces, e.g., 20–50 components.
- **Lightweight face detector:** Use a computationally efficient detector instead of a heavy deep-learning model.
- **Process only the detected face:** Avoid processing the entire image repeatedly.
- **Precompute Eigenfaces:** Store the PCA projection matrix and training feature vectors beforehand.
- **Fast distance calculation:** Use Euclidean distance between the projected feature vectors.
- **Use integer/fixed-point operations** where possible on embedded hardware.
- **Limit comparisons:** Organize stored face templates efficiently to reduce the number of comparisons.
- **Hardware acceleration:** If available, use the device's DSP/NPU/GPU for image-processing operations.

Overall approach

Camera → Face Detection → CLAHE + Gamma Correction → Denoising → Face Alignment → 64×64 Resize → PCA/Eigenfaces → Fast Distance Matching → Identity

Conclusion:

An Eigenface-based system can be made suitable for a low-light embedded environment by using illumination normalization, pose alignment, multiple pose templates, dimensionality reduction, and lightweight computation. These techniques improve recognition accuracy while helping the system meet the <200 ms real-time requirement.

Scenario 2: Photo Album Organization

An AI system is designed to automatically organize a large personal photo collection based on people and events.

1. Handling millions of images with different resolution and quality

- **Image preprocessing:** Resize all images to a standard size and normalize pixel values.
- **Quality enhancement:** Apply denoising, brightness correction, and contrast enhancement to poor-quality images.
- **Face detection:** Detect faces in each image using a lightweight face detector.
- **Feature extraction:** Extract a compact facial feature vector (embedding) for each detected face.
- **Efficient storage:** Store only the extracted features and metadata for fast searching instead of repeatedly processing original images.
- **Batch processing:** Process images in batches to handle millions of photos efficiently.

2. Handling different poses and multiple photos per person

- Use a face recognition model that is robust to changes in pose, expression, and lighting.
- Store multiple feature vectors for each person from different poses.
- Face alignment using facial landmarks helps normalize tilted or rotated faces.
- Group similar face embeddings using clustering techniques such as K-means or DBSCAN.
- Use a similarity measure such as cosine similarity to identify whether two faces belong to the same person.

3. Organizing photos by events

Event information can be obtained from:

- Date and time from image metadata (EXIF).
- Location/GPS information, if available.
- Similar visual content, such as birthday parties, weddings, trips, or festivals.
- Photos taken close together in time and location can be grouped into the same event.

4. Privacy considerations

Since all processing must occur locally:

- No images should be uploaded to cloud servers.
- Store face embeddings and personal information securely on the local device.
- Use encryption for stored photos and feature data.
- Restrict access using authentication and permissions.
- Delete temporary processing files after use.
- Use an offline AI model so that face recognition works without an Internet connection.

Overall System

Photos → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Face Clustering → Person Identification → Event Detection → Organized Albums

Conclusion

The system should use local face detection and recognition, clustering, metadata-based event detection, and efficient feature storage. Batch processing and compact embeddings help handle millions of images, while offline processing and encryption protect user privacy.

Scenario 3: Smart City Surveillance System

A smart city surveillance system needs to detect moving objects and track pedestrians across multiple cameras.

1. Foreground–Background Separation

To minimize false positives in crowded scenes, use an adaptive background subtraction method.

Steps:

1. Build a background model using several frames.
2. Compare the current frame with the background model.
3. Apply adaptive thresholding to detect foreground objects.
4. Use morphological operations such as opening and closing to remove noise.
5. Apply connected-component analysis to identify actual objects.
6. Continuously update the background model so that gradual lighting changes do not create false detections.
7. Use a person detector to verify that detected regions are pedestrians.

Flow:

Video → Background Model → Foreground Mask → Noise Removal → Person Detection → Pedestrian Objects

This combination reduces false positives caused by shadows, lighting changes, and background movement.

2. Particle Filters for Occlusion Handling

A particle filter represents the possible position of a pedestrian using many particles.

Working:

1. Initialize particles around the detected pedestrian.
2. Predict the pedestrian's next position using previous motion.
3. Assign weights to particles according to how well they match the current image.
4. During occlusion, visual information may disappear, but the particles continue predicting the person's likely position.
5. When the pedestrian becomes visible again, the particles are updated using the new observation.
6. Resampling keeps the particles with higher probabilities.

Example:

Person detected → Particles generated → Person becomes occluded → Predict position → Person reappears → Update particles → Continue tracking

Thus, particle filters can maintain tracking even when a pedestrian is temporarily hidden by another person, vehicle, or object.

3. Efficient Multi-Camera Pedestrian Identification

Use a distributed tracking and re-identification (Re-ID) approach.

- Each camera performs local person detection and tracking.
- Instead of sending complete video, cameras send only compact features/embeddings, object ID, timestamp, and location.
- Extract appearance features such as clothing color, body shape, and visual descriptors.
- Compare embeddings between cameras to determine whether two detections belong to the same person.
- Use camera timestamps and spatial information to reject unlikely matches.
- Maintain a global pedestrian ID across cameras.

Example:

Camera 1 → Person ID 15 → Feature Vector → Network → Camera 2 →
Matching → Same Global ID 15

Overall Architecture

Camera 1: Detection + Tracking + Feature Extraction



Compact Feature Transmission



Central/Edge Server: Re-ID + Multi-camera Association



Global Pedestrian Tracking

This approach reduces network bandwidth because raw video does not need to be continuously transmitted, while local processing helps satisfy the real-time requirement.

Scenario 4: Self-Driving Car – Urban Road Detection

1. Road detection under shadows, rain, and night conditions

To make road detection robust to different weather and lighting conditions, use a combination of image preprocessing, color analysis, edge detection, and adaptive thresholding.

Steps:

1. **Image enhancement** – Improve dark or low-contrast images using histogram equalization or CLAHE.
2. **Noise removal** – Apply a Gaussian/bilateral filter to reduce rain and sensor noise.
3. **Color space conversion** – Convert RGB to **HSV or HLS** to separate road color from illumination.
4. **Shadow handling** – Use brightness and color information to distinguish shadows from actual road boundaries.
5. **Lane detection** – Apply **Canny edge detection** followed by Hough Transform to detect lane markings.
6. **Adaptive processing** – Adjust thresholds according to brightness and weather conditions.
7. **Temporal information** – Compare consecutive frames to maintain stable road and lane detection.

Flow:

Camera → Enhancement → Noise Removal → HSV/HLS → Shadow/Rain Handling → Edge Detection → Lane/Road Detection

2. Efficient pipeline for road signs and pedestrians

A single efficient vision pipeline can detect both road signs and pedestrians:

Camera Frame → Preprocessing → Region Detection → Object Detector → Classification → Tracking → Decision

Steps:

1. Capture the camera frame.
2. Resize and normalize the image.

3. Use a lightweight object detection model to detect pedestrians and road signs.
4. Classify detected objects into:
 - Pedestrian
 - Stop sign
 - Speed-limit sign
 - Traffic light, etc.
5. Use object tracking to track detected pedestrians and signs across frames.
6. Estimate their position and movement.
7. Send the results to the vehicle's decision-making system.

For efficiency, use region-of-interest (ROI) processing and process only important areas of the image.

3. Deep Learning vs Classical Computer Vision

Factor	Deep Learning	Classical Computer Vision
Accuracy	High	Moderate
Weather/lighting robustness	Better after training	Usually lower
Complex objects	Handles well	Difficult
Computation	High	Low
Training required	Yes	No/less
Real-time on powerful hardware	Good	Excellent
Embedded hardware	May require optimization	Easier to run

Trade-off

Deep learning methods such as YOLO-based detectors provide better accuracy for pedestrians and road signs, especially in crowded scenes and different viewpoints. However, they require more computational power.

Classical methods such as Canny, Hough Transform, color thresholding, and Haar-like features are faster and computationally cheaper, but they are more sensitive to rain, shadows, illumination, and object variations.

Best approach

A hybrid approach is preferable:

- Use classical computer vision for simple tasks such as lane/edge detection.
- Use a lightweight deep-learning detector for pedestrians and road signs.
- Use GPU/NPU acceleration and reduced image resolution to achieve the ≤ 50 ms/frame requirement.

Conclusion: A hybrid system provides a good balance between accuracy, robustness, and real-time performance for an urban self-driving car.

Scenario 5: AR System for Surgical Assistance

An AR surgical-assistance system must segment organs from CT/MRI images and overlay them in real time.

1. Real-time organ segmentation under memory and latency constraints

Use a lightweight 3D segmentation network and optimize the input data.

Steps:

1. **Preprocess CT/MRI images** – Normalize intensity and remove unnecessary noise.
2. **Crop the Region of Interest (ROI)** – Process only the area containing the required organ instead of the complete 3D volume.
3. **Downsample the volume** – Reduce image resolution while preserving important anatomical information.
4. Use a **lightweight 3D CNN/U-Net** for organ segmentation.
5. Use **patch-based processing** so that only small portions of the volume are processed at a time.
6. Use **GPU memory optimization**, such as mixed-precision computation (FP16).
7. Keep the segmentation model and frequently used data in memory to avoid repeated loading.

Flow:

CT/MRI → Preprocessing → ROI Selection → Downsampling → Lightweight 3D U-Net → Segmentation → AR Overlay

This reduces both memory usage and processing latency.

2. Accurate integration of segmentation into AR/MR overlays

The segmented organ must be correctly aligned with the patient's actual anatomy.

Important steps:

- **Image registration:** Align CT/MRI coordinates with the patient's physical body.
- **Camera/pose tracking:** Track the position and orientation of the AR headset.
- **Spatial calibration:** Calibrate the headset and surgical environment.
- **Depth information:** Use depth sensors to estimate the patient's surface.
- **3D transformation:** Convert the segmented organ coordinates into the headset's coordinate system.
- **Continuous tracking:** Update the overlay as the surgeon or patient moves.
- Use semi-transparent visualization so that the surgeon can see both the real anatomy and the segmented organ.

Flow:

CT/MRI Segmentation → Registration → Coordinate Transformation → Headset Tracking → 3D Overlay

Accurate registration is essential because even a small alignment error can result in an incorrect surgical overlay.

3. Reducing computational load while maintaining accuracy

The following techniques can reduce computation:

1. **ROI processing** – Process only the relevant organ region.
2. **Model compression** – Use pruning and quantization to reduce model size.
3. **Knowledge distillation** – Train a small model using a larger accurate model.

4. **Mixed precision (FP16)** – Reduce memory consumption and computation.
5. **Lower input resolution** – Use an appropriate resolution instead of the full CT/MRI resolution.
6. **Frame skipping** – Avoid recomputing segmentation when the scene has not changed significantly.
7. **Temporal tracking** – Propagate segmentation from previous frames and update only when necessary.
8. **Lightweight architecture** – Use a compact U-Net or MobileNet-based segmentation network.
9. **Hardware acceleration** – Use the headset's available GPU/NPU efficiently.

Conclusion

A lightweight 3D segmentation model + ROI processing + model compression + mixed-precision computation can satisfy the memory and latency constraints. Accurate registration, spatial calibration, and headset tracking ensure that the segmented organs are correctly aligned with the real patient, enabling reliable real-time AR surgical assistance.